

**The AI agent revolution has a dirty secret:** we're building autonomous systems with the permission models of a 1990s file server. While we obsess over prompt engineering and tool calling, the fundamental question of *what agents are permitted to do* —and more importantly, *why* —remains governed by ad-hoc JSON schemas and vibes-based access control.

I've spent decades working at the intersection of legal informatics and software architecture. My early work on graphical notations for legal norms using deontic logic and Hohfeldian analysis wasn't just academic exercise—it was building formal tools for reasoning about rights, duties, and permissions. Three decades later, these same tools are exactly what we need to bring rigor to AI agent governance.

The **\*\*Model Context Protocol\*\*** (MCP) represents a significant step forward in standardizing agent-tool interaction, but its authorization model inherits the same conceptual poverty that plagues the rest of the industry.

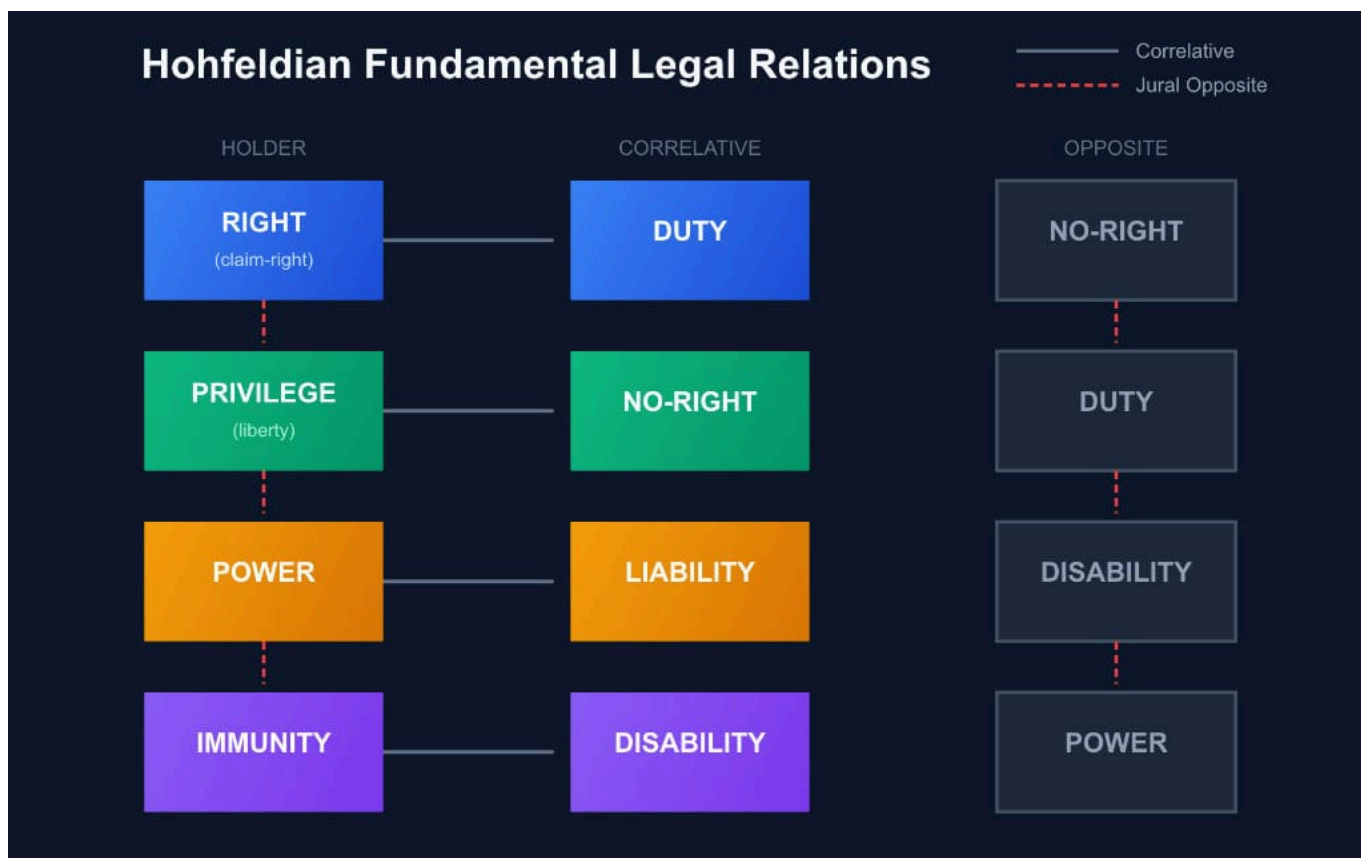
Let's fix that.

## The Hohfeldian Framework: Legal Relations as Permission Primitives

[Wesley Newcomb Hohfeld](#), writing in the early 20th century, performed one of the most important analytical feats in legal philosophy: he decomposed the vague concept of "rights" into eight fundamental legal relations. This taxonomy has stood the test of a century of legal analysis because it captures something true about the structure of normative relationships.

### The Eight Fundamental Relations

Hohfeld identified four pairs of correlative relations:



The genius here is recognizing that these aren't just abstract categories—they're *computationally meaningful*. Every permission statement can be decomposed into these primitives, and every permission *interaction* follows predictable algebraic rules.

**Right-Duty Correlation:** If Agent A has a *right* that Agent B perform action X, then Agent B has a correlative *duty* to perform X. This isn't philosophy—it's a constraint that must be enforced at runtime.

**Privilege-NoRight Correlation:** If Agent A has a *privilege* to perform action X, no other agent has a *right* that A not perform X. Privileges are permission-without-obligation—the agent *may* act but need not.

**Power-Liability:** The most important relation for agent systems. If Agent A has *power* to change Agent B's normative position (grant permissions, revoke access, modify capabilities), then B is *liable* (under a *liability*) to have its position changed. This is delegation, authorization, and governance in formal dress.

**Immunity-Disability:** The inverse of power. If Agent A has *immunity* from Agent B's attempted normative changes, then B is under a *disability* —it lacks the power to affect A's position.

## Why This Matters for Agents

Consider a typical MCP scenario: an agent needs to read a file, modify a database, and send an email.

Current authorization asks: "Does the agent have permission?"

The Hohfeldian analysis asks something richer:

- Does the agent have a **privilege** to read (it may read, but needn't)?
- Does the agent have a **right** to read (some other entity has a duty to provide access)?
- Does the agent have a **power** to grant sub-agents read access?
- Does the agent have **immunity** from having its read access revoked during execution?

These distinctions aren't academic. They determine failure modes, recovery strategies, and accountability chains.

## Deontic Logic: From Relations to Calculus

[Deontic logic](#) formalizes normative concepts—obligation, permission, prohibition—using modal operators. The standard operators are:

- **O(p)**: It is obligatory that p
- **P(p)**: It is permitted that p
- **F(p)**: It is forbidden that p (equivalent to  $O(\neg p)$ )

The symbol  **$\neg$**  is the logical **negation** operator, commonly read as "not." Meaning if

P

is a proposition, then

$\neg P$

(or

$\neg P$

) is the proposition that is true when

P

is false, and false when

P

is true. It flips the truth value. So, if

P

represents "It is raining," then

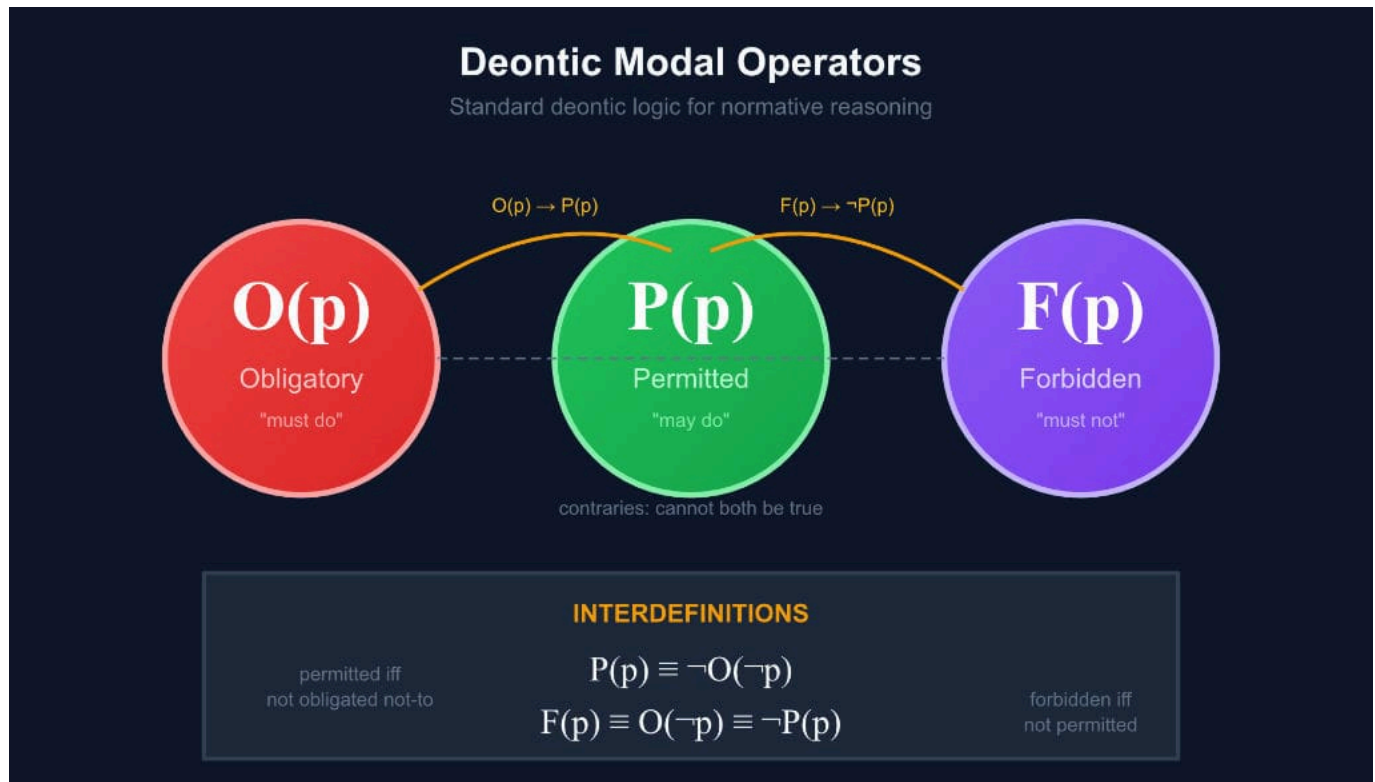
$\neg P$

means "It is **not** raining."

With the standard inter-definition:  $P(p) \equiv \neg O(\neg p)$  and  $F(p) \equiv \neg P(p)$ .

These three concepts are connected: Here's a plain-English version:

If something is **allowed**, it means you **don't have to avoid** it. If something is **forbidden**, it means it's **not allowed**.



## The Permission Calculus for Agents

Let's build a concrete permission calculus.

Define:

**Agent**:= identifier

**Resource**:= identifier

**Action**:= read | write | execute | delegate | revoke

**Scope**:= temporal\_bound × resource\_bound × context\_bound

**Permission**:= (agent, action, resource, scope, provenance)

The key insight is that permissions aren't binary—they're structured objects with:

1. **Provenance**: Who granted this permission? Under what authority?

2. **Scope:** What are the temporal, resource, and contextual bounds?
3. **Delegation chain:** Can this permission be transferred? To whom? With what constraints?

## Obligation Chains in Multi-Agent Systems

When Agent A delegates to Agent B, we create an *obligation chain*:

$\text{delegate}(A, B, \text{permission}(\text{read}, R, S)) \rightarrow$   
 $O(A, \text{audit}(B.\text{actions}(R))) \wedge$   
 $O(A, \text{revoke}(B, R) \mid \text{violation}(B, R)) \wedge$   
 $\text{liable}(A, \text{damages}(B.\text{actions}(R)))$

### In plain English:

When A delegates read access on resource R to B within scope S,  
A becomes obligated to audit B's actions  
A is obligated to revoke access upon violation, and  
A is liable for damages arising from B's actions.

This is accountability with teeth.

## The Closure Problem

Deontic systems must also address the *closure problem*: what is the normative status of actions not explicitly addressed?

Two approaches:

**Permissive closure:** Everything not forbidden is permitted.

**Prohibitive closure:** Everything not permitted is forbidden.

For agent systems, the answer is contextual:

- A general-purpose assistant might operate under permissive closure with explicit prohibitions.
- A financial trading agent operates under prohibitive closure with explicit permissions

The MCP authorization model needs to express this distinction.

## Mapping Hohfeld's Framework to MCP Authorization

The [Model Context Protocol](#) defines tools, resources, and prompts. Let's map the Hohfeldian relations onto this structure.

---

## MCP Authorization: Current vs. Hohfeldian Model

### CURRENT MODEL

Path-based ACL

```
{
  "tool": "file_read",
  "permissions": {
    "allowed_paths": [ "/data/*" ],
    "denied_paths": [ "/data/secrets/*" ]
  }
}
```

#### WHAT'S MISSING

- Who authorized this permission?
- Under what policy/authority?
- Can the agent delegate to sub-agents?
- What obligations follow from access?
- Who is liable if something goes wrong?

### HOHFELDIAN MODEL

Formal Permission Calculus

#### RELATION TYPE

privilege | right | power | immunity



#### PROVENANCE

grantedBy: AgentIdentifier

authority: PolicyReference



#### SCOPE

temporal: validFrom → validUntil

context: ExecutionContext[]



#### DELEGATION

delegable: boolean

constraints: DelegationConstraint[]



#### CORRELATIVE OBLIGATIONS

audit, backup, rate-limit duties



#### LIABILITY CHAIN

liabilityChain: AgentIdentifier[]



## Current MCP Authorization (Simplified)

```
{
  "tool": "file_read",
  "permissions": {
    "allowed_paths": [ "/data/*" ],
    "denied_paths": [ "/data/secrets/*" ]
  }
}
```

This is path-based ACL thinking. It tells us *what* but not *why*, *who authorized it*, *under what conditions*, or *what happens when things go wrong*.

## Hohfeldian MCP Authorization (Proposed)

```
interface HohfeldianPermission {
  // The normative relation type
  relation: 'right' | 'privilege' | 'power' | 'immunity';

  // The correlative obligation (for rights) or liability (for powers)
  correlative?: {
    bearer: AgentIdentifier;
    content: Obligation | Liability;
  };

  // What action on what resource
  action: Action;
  resource: ResourcePattern;
}
```

```

// Temporal and contextual bounds
scope: {
  validFrom: Timestamp;
  validUntil: Timestamp;
  contexts: ExecutionContext[];
  conditions: Condition[];
};

// Provenance chain
grantedBy: AgentIdentifier;
grantedUnder: AuthorityReference;
delegable: boolean;
delegationConstraints?: DelegationConstraint[];

// Accountability
auditRequirements: AuditSpec;
liabilityChain: AgentIdentifier[];
}

```

## The Permission DSL

The natural next step is to define a domain-specific language (DSL) for expressing agent permissions. Here's a simple example using a straight-forward declarative syntax:

```

// Permission declaration
permission FileReadPrivilege {
  relation: privilege
  agent: DataAnalysisAgent
  action: read
  resource: /data/analytics/**

  scope {
    valid: 2024-01-01 to 2024-12-31
    context: [production, staging]
    condition: request.purpose in ["reporting", "analysis"]
  }

  granted_by: SystemAdmin
  authority: DataGovernancePolicy.section_3_2
  delegable: false
}

// Delegation chain
delegation AnalyticsToVisualization {
  from: DataAnalysisAgent
  to: VisualizationAgent
  permission: FileReadPrivilege
}

```

```

constraints {
  scope_restriction: resource = /data/analytics/charts/**
  temporal_restriction: duration <= 1 hour
  audit: full_trace
}

obligations {
  delegator_must: [audit_access, revoke_on_anomaly]
  delegatee_must: [log_all_reads, respect_rate_limits]
}
}

// Power declaration
power DelegateReadAccess {
  relation: power
  holder: TeamLeadAgent

  // What normative changes can be made
  can_create: privilege where {
    action in [read]
    resource matches /team_data/**
  }

  // Who is liable to these changes
  liability_bearers: [TeamMemberAgent]

  // Limits on power exercise
  constraints {
    max_delegation_depth: 2
    requires_approval_above: 10 resources
  }
}

// Immunity declaration
immunity CoreSystemProtection {
  relation: immunity
  holder: AuditAgent

  // What powers cannot affect this agent
  immune_from: [
    revoke_access where granted_by = SystemAdmin,
    modify_permissions where resource = /audit/**
  ]

  // Who is under correlative disability
  disabled_agents: [*, except: SuperAdmin]
}

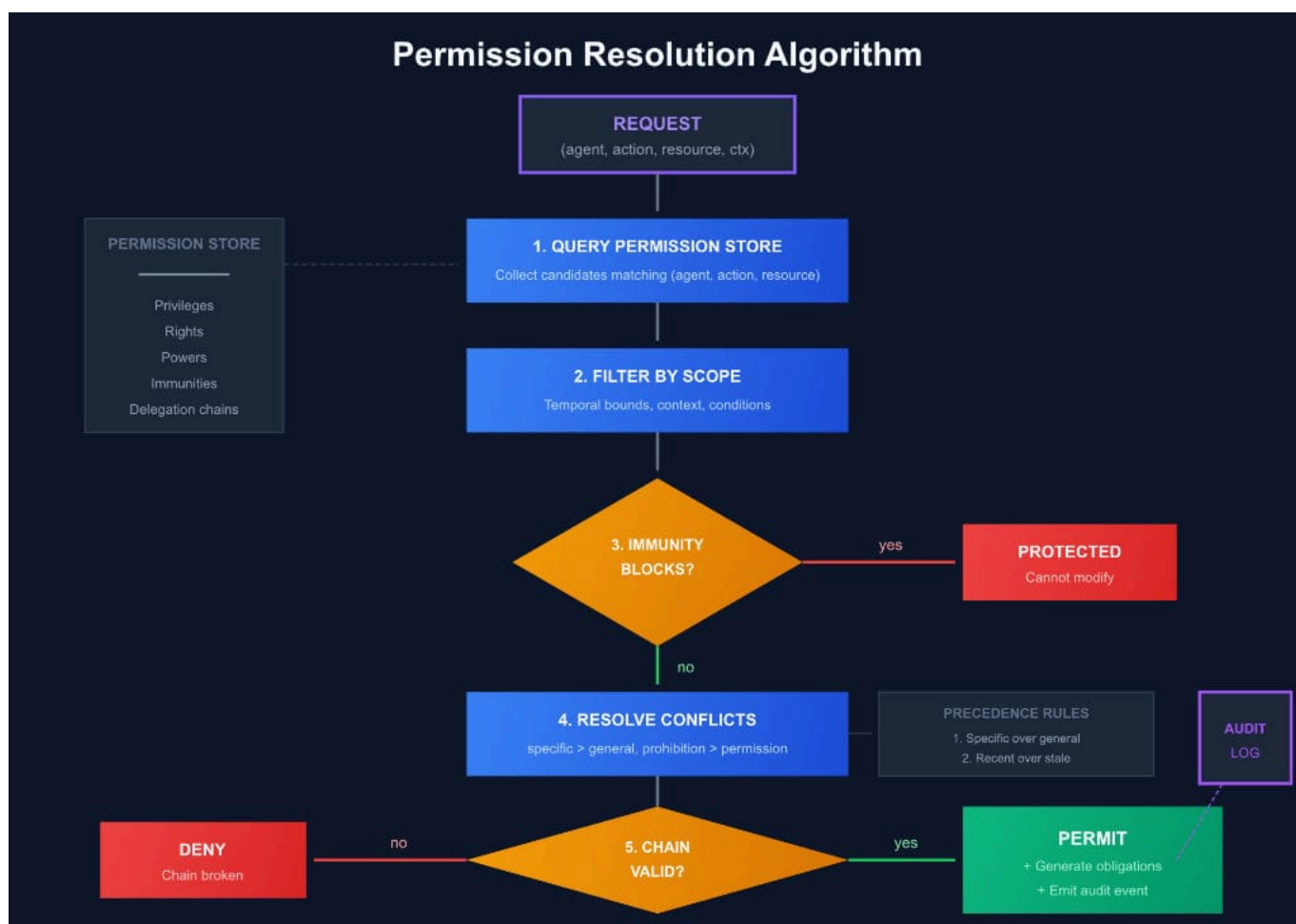
```

This DSL makes several things explicit that are currently implicit or absent in MCP agent authorization:

1. **Relation type** distinguishes can-do (privilege) from must-enable (right) from can-change (power) from cannot-be-changed (immunity).
2. **Provenance** establishes who authorized what under which policy—essential for audit and accountability.
3. **Delegation constraints** specify exactly how permissions can flow through agent hierarchies.
4. **Correlative obligations** automatically generate the duty-side of rights and the liability-side of powers.

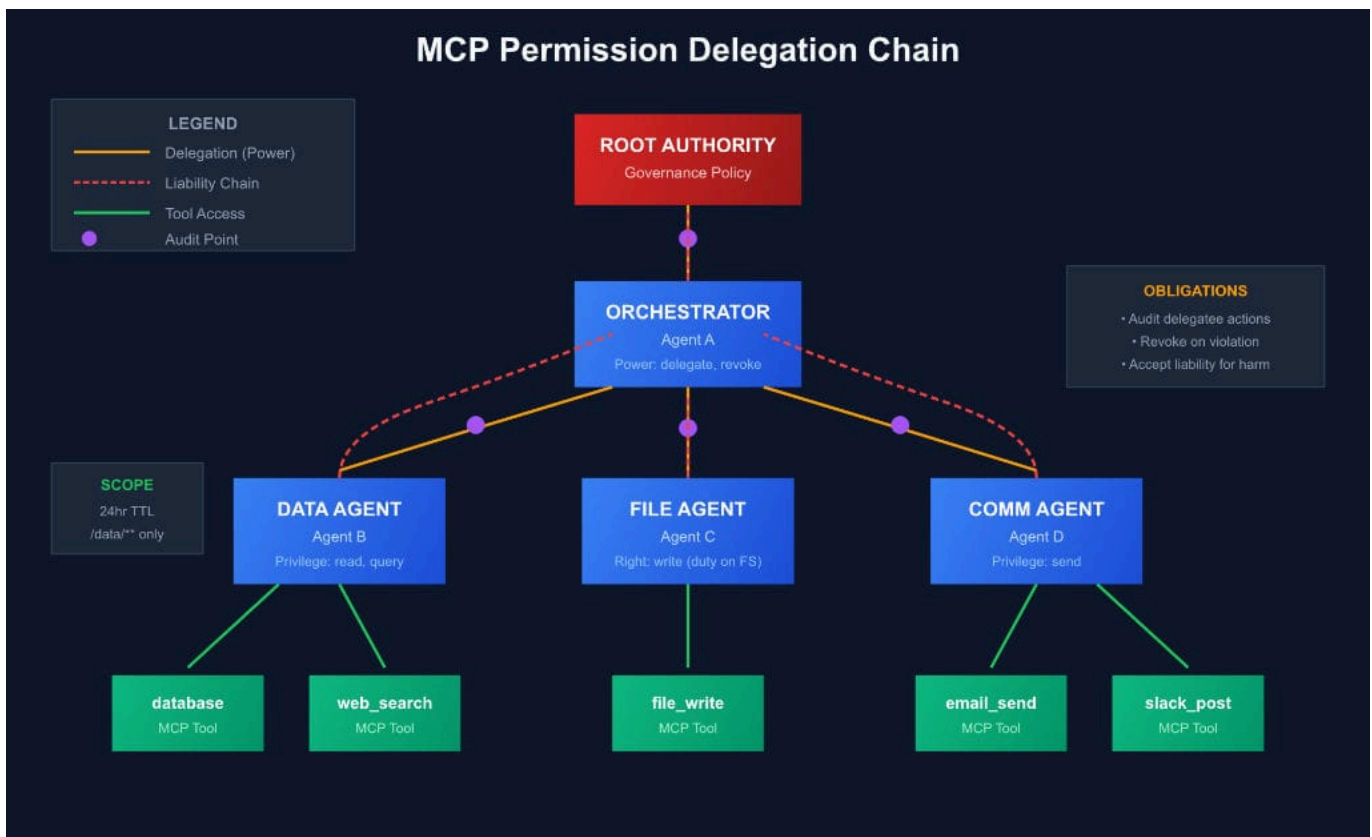
## Operational Semantics: Making It Run

A formal framework is useless if it can't be evaluated at runtime. Here's the operational semantics for permission checking:



## Delegation Chain Validation

The most complex operation is validating delegation chains:



## Governance Implications: Accountability When Agents Fail

The formal framework enables precise answers to governance questions that are currently hand-waved:

### Who Owns Agent Behavior?

The liability chain in each permission explicitly specifies ownership. When Agent C acts under a permission delegated from B who received it from A:

**liability\_chain: [C, B, A]**

Damage assessment follows this chain. If C causes harm:

- C is immediately responsible for the action
- B is responsible for inadequate oversight (violated audit obligation)
- A is responsible for delegation policy failure

### How are decisions reviewed?

Every permission carries audit requirements:

```
interface AuditSpec {
  granularity: 'action' | 'session' | 'daily' | 'on_anomaly';
  retention: Duration;
  reviewers: AgentIdentifier[];
```

```
escalation: EscalationPolicy;
}
```

The audit trail automatically captures:

- What permission was exercised
- Under what authority (full provenance chain)
- What obligations were generated
- Whether those obligations were satisfied

## What recourse exists?

The power/immunity relations define recourse explicitly:

1. **Revocation path:** Who has power to revoke which permissions?
2. **Immunity barriers:** What cannot be revoked even by administrators?
3. **Escalation:** When does automated governance yield to human review?

```
recourse_policy {
  // Automatic revocation triggers
  auto_revoke when {
    violation_count > 3 in 24 hours
    anomaly_score > 0.9
    delegation_chain_invalidated
  }

  // Human review triggers
  escalate_to: HumanOversight when {
    action in [delete, modify] and resource.sensitivity = "high"
    cumulative_cost > $10000
    user_complaint
  }

  // Protected operations (immunity applies)
  protected {
    audit_logs: immune_from revocation by [*, except: LegalHold]
    safety_monitors: immune_from modification by [*, except: SafetyBoard]
  }
}
```

## Implementation Path: From Theory to Practice

### Phase 1: Permission Annotation

Add Hohfeldian annotations to existing MCP tool definitions:

```
const tool = {
  name: "file_write",
  // Existing MCP definition
  inputSchema: { /* ... */ },

  // Hohfeldian extension
  hohfeld: {
    default_relation: "privilege",
    requires_power_for_delegation: true,
    generates_obligations: [
      { type: "audit", granularity: "action" },
      { type: "backup", before_destructive_write: true }
    ]
  }
};
```

## Phase 2: Permission DSL

Implement the permission DSL.

## Phase 3: Runtime Integration

Build a permission resolution service that:

1. Parses permission DSL files
2. Maintains the permission store with efficient indexing
3. Provides the `resolvePermission` API
4. Generates audit events
5. Enforces obligation chains

## Phase 4: Governance Dashboard

Surface the formal model in human-understandable terms:

- Visualize delegation chains
- Alert on permission anomalies
- Support delegation approval workflows
- Enable "what-if" permission analysis

## Conclusion: Formal Methods for Practical Governance

The AI agent ecosystem is building increasingly powerful autonomous systems on governance foundations of sand. We've been here before—in the early days of computing, access control was similarly ad-hoc until formal models ([Bell-LaPadula](#), [Biba](#), [Clark-Wilson](#)) brought rigor to security policy.

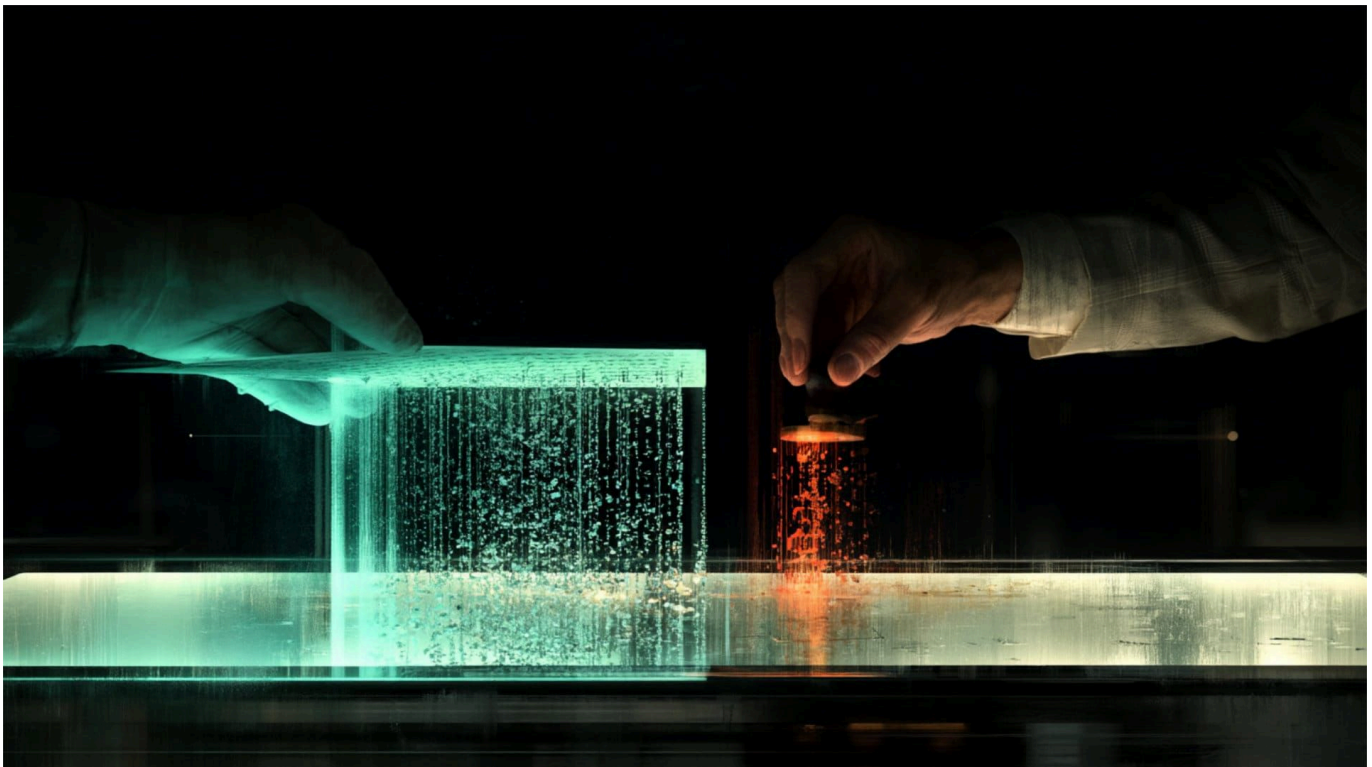
Hohfeldian analysis and deontic logic provide the same formal foundation for agent permissions. They're not just academic tools—they're operational frameworks that:

1. **Decompose** vague permission concepts into computationally tractable primitives
2. **Compose** complex authorization policies from well-understood building blocks
3. **Verify** delegation chains and obligation satisfaction
4. **Audit** with full provenance and accountability
5. **Reason** about permission interactions and conflicts

The [Model Context Protocol](#) is the right foundation for standardizing agent-tool interaction. Adding a formal permission calculus—grounded in a century of legal analysis—transforms it from a communication protocol into a governance framework.

The industry desperately needs this. Let's build it.

**Written by**



Two Algorithms, Zero Shared Memory